

Для начала надо пройти tutorial, предложенный на сайте.

Конфигурационный файл пакера определяет какой image мы хотим создать и каким образом его создать.

Для пакера нужно использовать файл с расширением .pkr.hcl, где hcl это Hashicorp Configuration Language.

```
packer {
  required_plugins {
    docker = {
      version = "≥ 1.0.8"
      source  = "github.com/hashicorp/docker"
    }
  }
}

source "docker" "ubuntu" {
  image = "ubuntu:jammy"
  commit = true
}

build {
  name      = "learn-packer"
  sources = [
    "source.docker.ubuntu"
  ]
}
```

Блок packer{} описывает его конфигурацию. В данном случае описана, какой нужен плагин, его версия и откуда его брать. Источник указывается, если плагина нет в домене корпорации Hashicorp. Также можно указать версию пакера.

Список плагинов можно найти тут

<https://developer.hashicorp.com/packer/integrations>

Блок source нужен для указания пакеру, какой билдер использовать(нужен для указания какую виртуализацию использовать) и имя. Также нужно указать в этом блоке как установить соединение с виртуальной машиной, golden image которой будем делать. Сам плагин ответственен за создание виртуальной машины и создание из этой машины образа.

В блоке build указывается, что делать с контейнером после его запуска из image.

Запускаем пакер. Он скачает нужный плагин.

```
user@arch ~/d/d/pack_tutorial (main)> packer init .  
Installed plugin github.com/hashicorp/docker v1.1.2 in "/home/user/.config/packer/plugins/github.com/hashicorp/docker/packer-plugin-docker_v1.1.2_x5.0_linux_amd64"
```

Перед билдом можно проверить template.

Форматирование:
packer fmt .

Проверка самой конфигурации:
packer validate .

```
user@arch ~/d/d/pack_tutorial (main)> packer fmt .  
docker-ubuntu.pkr.hcl  
user@arch ~/d/d/pack_tutorial (main)> packer validate .  
The configuration is valid.
```

Всё ОК, можно билдить.

Пакер, естественно, надо использовать под пользователем root и перед этим инициализировать тоже под этим пользователем. Иначе будет ошибка.

```
user@arch ~/d/d/pack_tutorial (main) [1]> sudo packer init .  
Installed plugin github.com/hashicorp/docker v1.1.2 in "/root/.config/packer/plugins/github.com/hashicorp/docker/packer-plugin-docker_v1.1.2_x5.0_linux_amd64"  
user@arch ~/d/d/pack_tutorial (main)> sudo packer build docker-ubuntu.pkr.hcl  
learn-packer.docker.ubuntu: output will be in this color.  
  
⇒ learn-packer.docker.ubuntu: Creating a temporary directory for sharing data.  
..  
⇒ learn-packer.docker.ubuntu: Pulling Docker image: ubuntu:jammy  
⇒ learn-packer.docker.ubuntu: jammy: Pulling from library/ubuntu  
  
⇒ Wait completed after 5 seconds 203 milliseconds  
  
⇒ Builds finished. The artifacts of successful builds are:  
→ learn-packer.docker.ubuntu: Imported Docker image: sha256:c850971811e1ff7348216ba4943d254c5c092a1e9bc9d6ce5d48d6f7387eb9df
```

Образ забилдился успешно. Проверяем.

```
216ba4943d254c5c092a1e9bc9d6ce5d48d6f7387eb9df
user@arch ~/d/d/pack_tutorial (main)> sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ubuntu:jammy	962f6cadeae0	117MB	31.7MB	

Всё успешно.

В следующем tutorialе ознакомимся со скриптами для редактирования образов докера. Темплейт использую тот же. Из документации:

Using provisioners allows you to completely automate modifications to your image. You can use shell scripts, file uploads, and integrations with modern configuration management tools such as Chef or Puppet.

Функционал, конечно, впечатляет.

```
build {
  name = "learn-packer"
  sources = [
    "source.docker.ubuntu"
  ]
  provisioner "shell" {
    environment_vars = [
      "F00=hello world",
    ]
    inline = [
      "echo Adding file to Docker Container",
      "echo \"F00 is $F00\" > example.txt",
    ]
  }
}
```

Устанавливаем переменную окружения, и выполняем команды в inline.

Итак, забыв о том, что ubuntu:jammy это base image, на основе которого делает итоговый image, несколько раз создал image. А не видно их было, потому что использовал docker images -a без параметра -a.....



В итоге пакер отработал как надо.

```
—> learn-packer.docker.ubuntu: Imported Docker image: sha256:42f11a29401835531df383c93f74d91e4cc34edcad13e1c14c66018a339ee2
user@arch ~/d/d/pack_tutorial (main)> nvim docker-ubuntu.pkr.hcl
user@arch ~/d/d/pack_tutorial (main)> sudo docker images -a
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ubuntu:jammy	962f6cadeae0	117MB	31.7MB	
<untagged>	42f11a294018	116MB	29.7MB	
<untagged>	808b01ed8a31	116MB	29.7MB	
<untagged>	cb6fd84afcb8	116MB	29.7MB	
<untagged>	ef7573a86d37	116MB	29.7MB	
<untagged>	c043dd50770e	116MB	29.7MB	
<untagged>	c850971811e1	115MB	29.7MB	

```
user@arch ~/d/d/pack_tutorial (main)> sudo docker run -it 42f11a294018
root@4523004cc3df:/# ls
bin  dev  home  lib32  libx32  mnt  packer-files  root  sbin  sys  usr
boot  etc  lib  lib64  media  opt  proc          run  srv  tmp  var
root@4523004cc3df:/# cat /tmp/example.txt
FOO is hello world
root@4523004cc3df:/#
```